

Design and Analysis of Algorithms

- *Course Code:* BCSE204L
- *Course Type:* Theory (ETH)
- *Slot:* A1+TA1 & & A2+TA2
- *Class ID:* VL2023240500901
VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 4

Module:4 | Graph Algorithms

6 hours

All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm
Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson,
Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to
maximum matching problem

Graph:

Graph is a non-linear data structures. That consists of following two components:

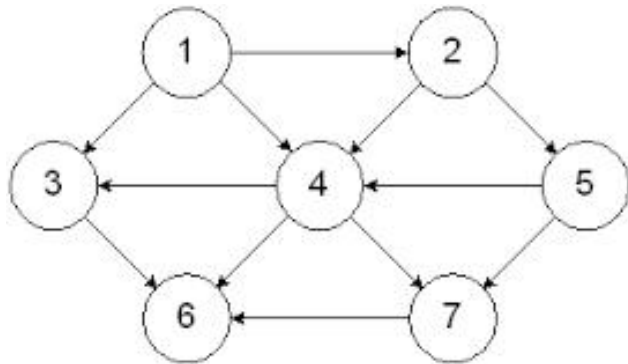
- A finite set of vertices also called nodes.
- A finite set of ordered pair of the form (u, v) , called as edge.

Graph $G=(V,E)$

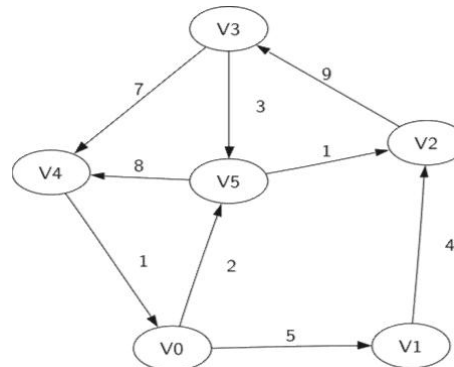
$V \rightarrow$ Vertices; $E \rightarrow$ Edges.

Edge is a pair (u,w) where (v,w) belongs to E

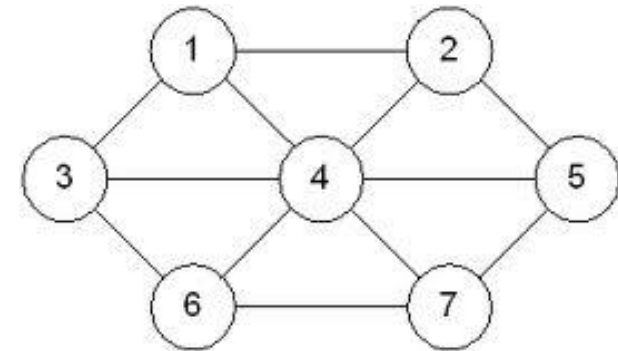
Directed Graph



Directed Weighted Graph

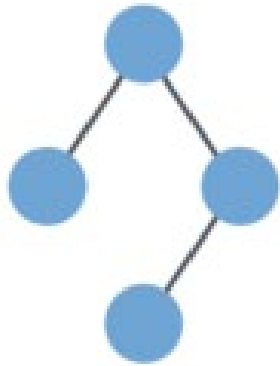


Undirected Graph

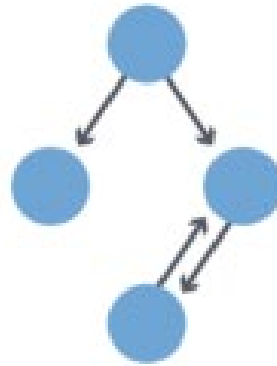


Graph type in general

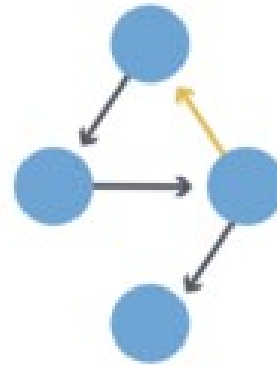
Undirected



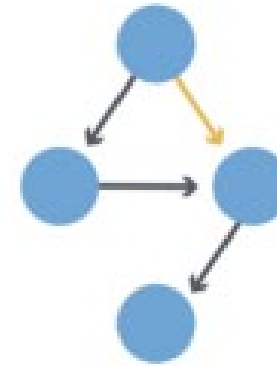
Directed



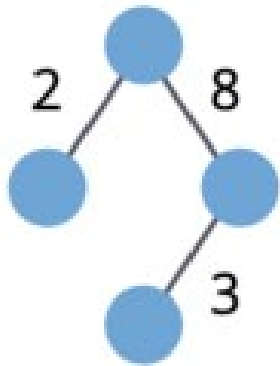
Cyclic



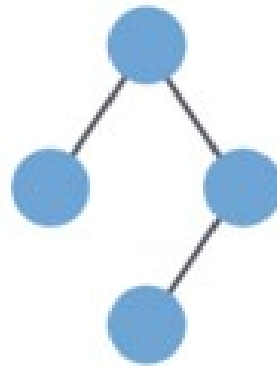
Acyclic



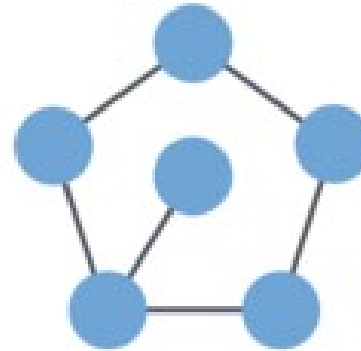
Weighted



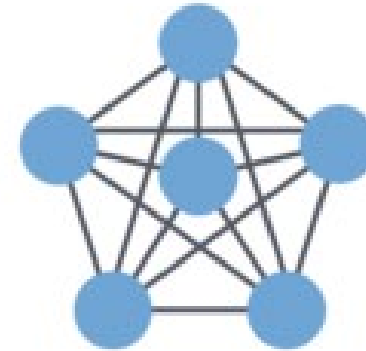
Unweighted



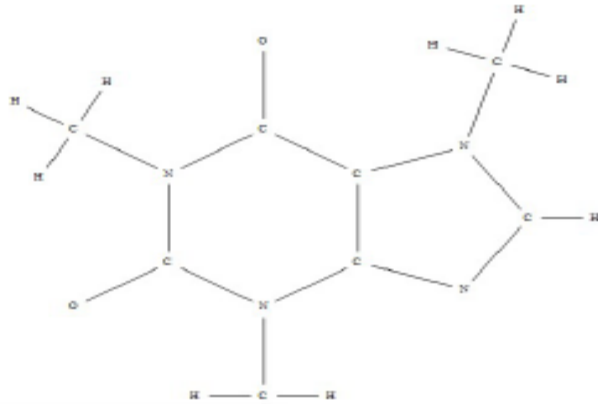
Sparse



Dense



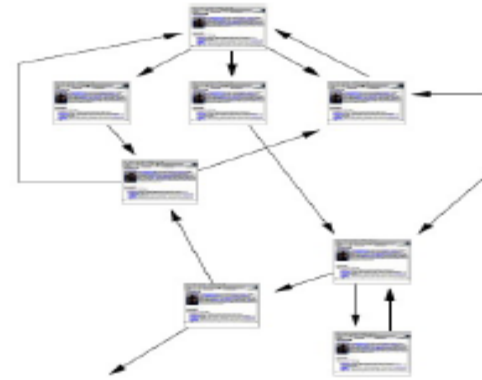
Graphs around us!



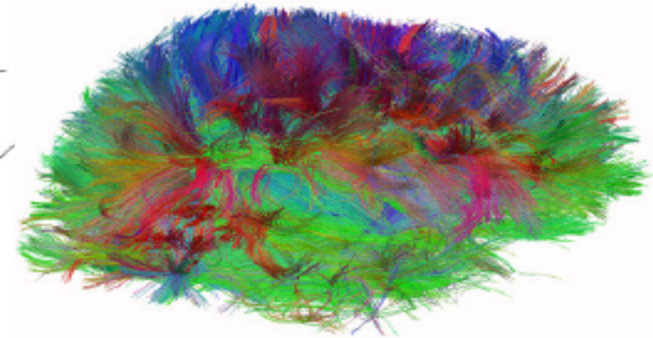
Molecules



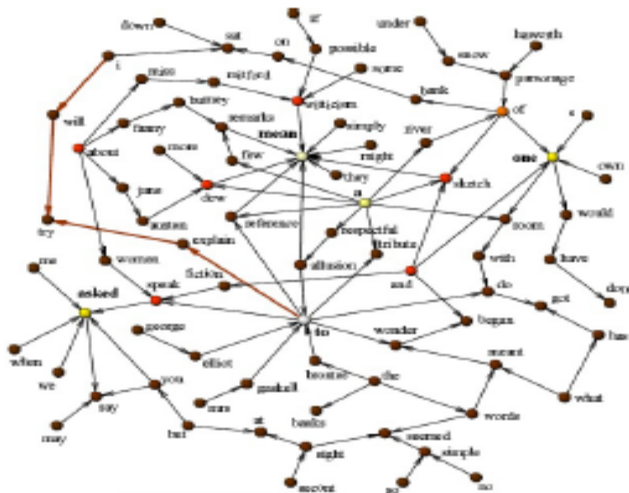
Knowledge



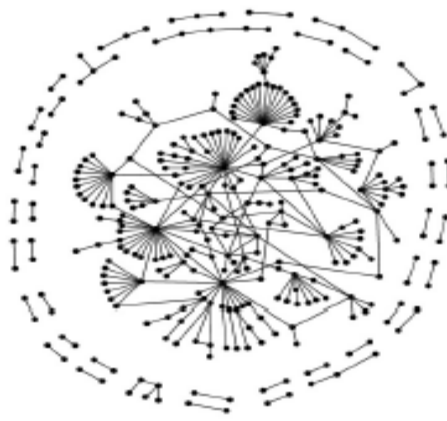
Information



Brain/neurons



Genes

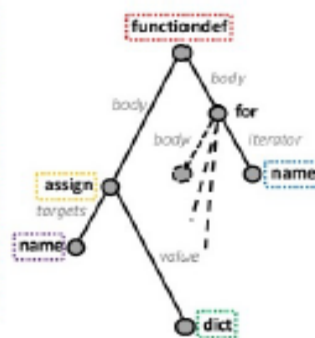


Communication

```
def encode(obj):
    """
    Encode a (possibly nested)
    dictionary containing complex value
    into a form that can be serialized
    using JSON.
    """
    e = {}
    for key, value in obj.items():
        if isinstance(value, dict):
            e[key] = encode(value)
        elif isinstance(value, complex):
            e[key] = {'type': 'complex',
                      'r': value.real,
                      'i': value.imag}
    return e
```

```
import ast
tree = ast.parse("""
```

Software



Social



Application of graphs

- Colouring of MAPS
- Representing network
 - Paths in a city
 - Telephone network
 - Electrical circuits etc.
- It is also using in social network including
 - LinkedIn
 - Facebook

Graph

- Graphs are a general language for describing and analysing entities with relations/interactions. Graphs can be used to represent most real-world data.
- A graph data structure has two basic elements: nodes and edges
 - *Nodes* represent entities in the data such as members of an online social network
 - *Edges* symbolise relationships between those entities, such as the friendship between members of a social network.
- Graphs are used almost in every field.
 - In *social networks*, graphs are used to provide online recommendations, implement newsfeed and calculate page rank.
 - In the field of *neuroscience*, neurons are denoted by nodes and connections between them as edges. These graphs are then used to analyze the functionality of brain networks.
 - In *chemical engineering*, covalent structures are represented as graphs. Hydrocarbon structure, protein structure are represented in the form of graphs in bioinformatics field.
- There are *many more applications* in other fields. These prove the importance of working with graphs.

Graph Terminology

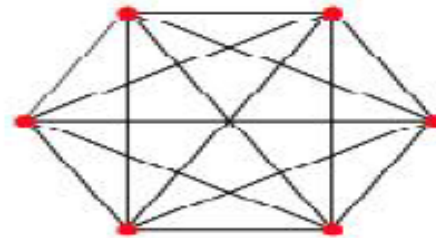
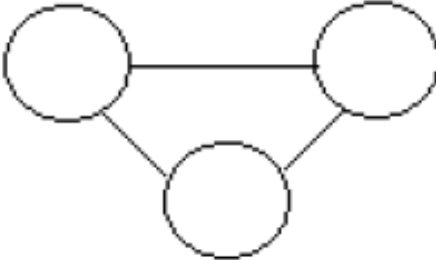
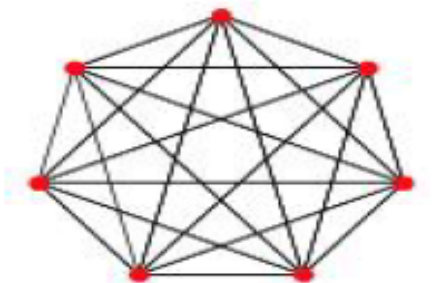
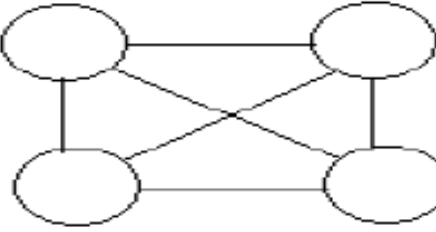
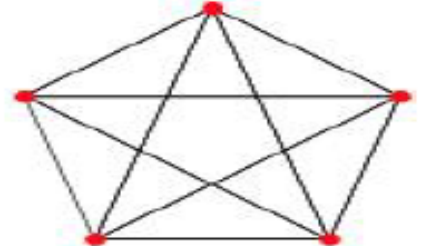
- The **order of a graph** is the **number of its vertices** $|V|$.
- The **size of a graph** is the **number of its edges** $|E|$.
- The **degree of a vertex** is the **number of edges that are adjacent to it**.
- The **neighbors of a vertex v** in a graph G is a subset of vertex V induced by all vertices adjacent to v .

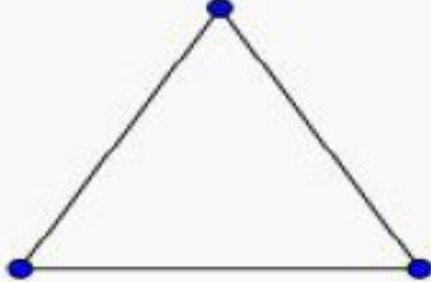
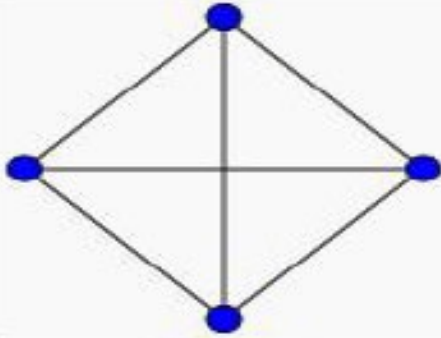
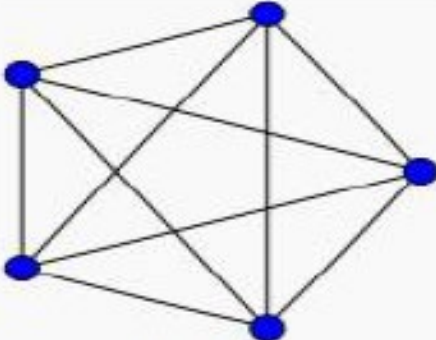
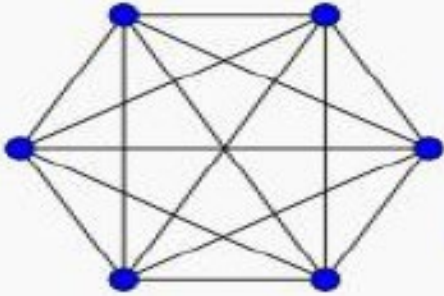
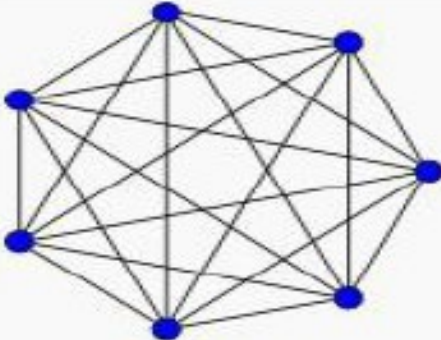
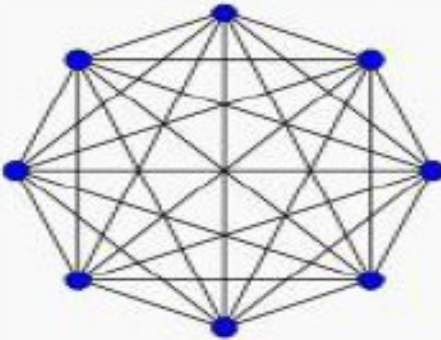
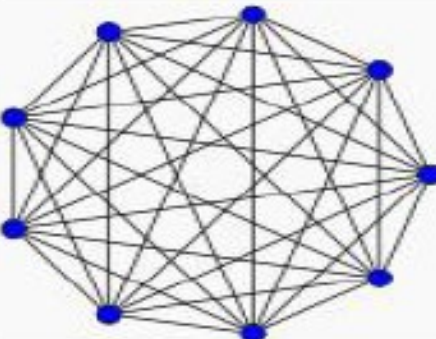
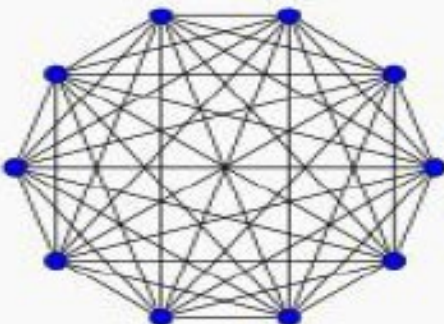
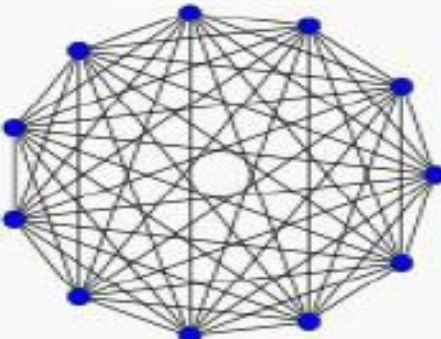
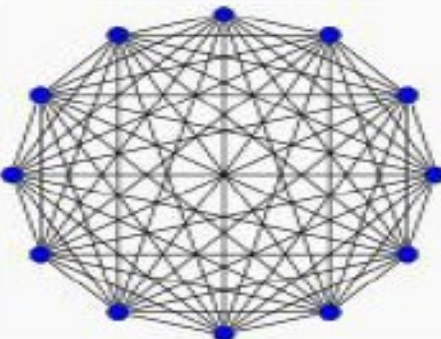
Complete Graph:

A **complete graph** is a simple undirected graph in which every pair of distinct vertices is connected by a unique edge.

(OR)

If an undirected graph of n vertices consists of $\frac{n(n-1)}{2}$ number of edges then the graph is called complete graph. **Example:**

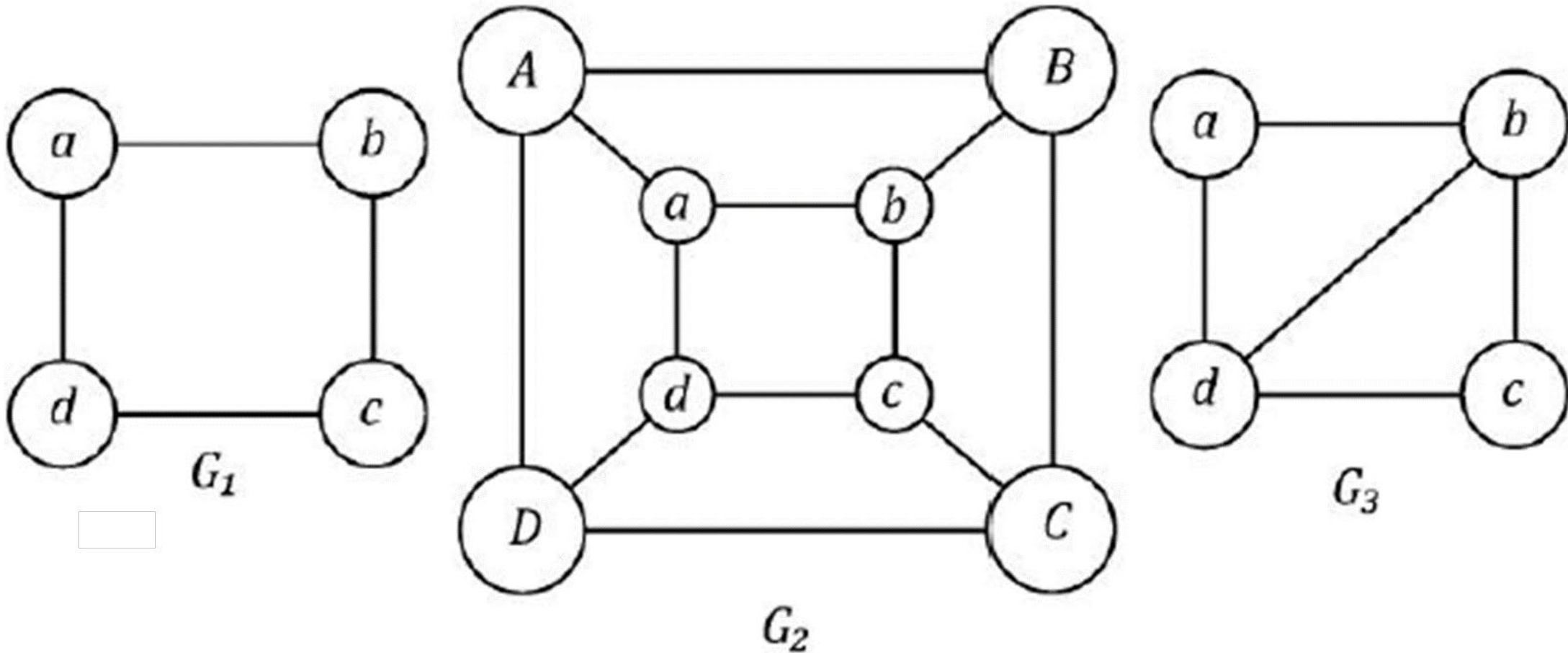
vertices	Edges	Complete graph	vertices	Edges	Complete graph
$n=2$	1		$n=6$	15	
$n=3$	3		$n=7$	21	
$n=4$	6		$n=5$	10	

$K_1: 0$	$K_2: 1$	$K_3: 3$	$K_4: 6$
			
$K_5: 10$	$K_6: 15$	$K_7: 21$	$K_8: 28$
			
$K_9: 36$	$K_{10}: 45$	$K_{11}: 55$	$K_{12}: 66$
			

n vertices
consists of
 $\frac{n(n-1)}{2}$
number of
edges

Sub graph:

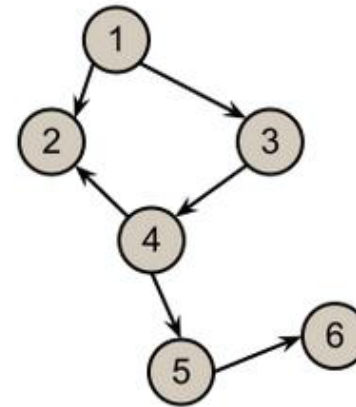
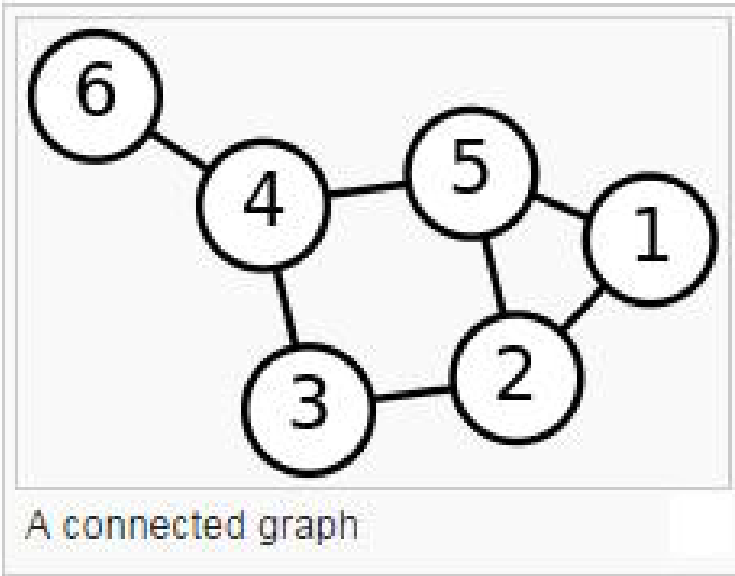
A sub-graph G' of graph G is a graph, such that the set of vertices and set of edges of G' are proper subset of the set of vertices and set of edges of graph G respectively.



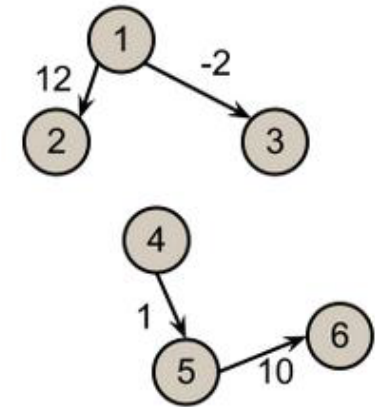
Connected Graph:

A graph which is connected in the sense of a topological space (study of shapes), i.e., there is a path from any point to any other point in the graph.

A graph that is not connected is said to be **disconnected**.



Connected



Disconnected

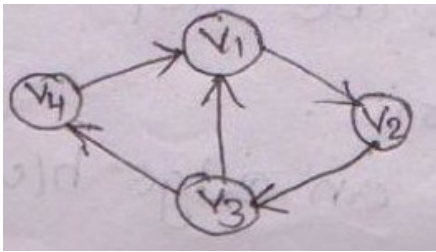
Representation of Graphs:

- There are two methods for graphs storage representations
 - Adjacency matrix
 - Adjacency List
- **Adjacency matrix:**
 - In which a two dimensional matrix is used for representing graph.
 - In this matrix
 - Rows $\rightarrow$ Source vertex
 - Columns $\rightarrow$ Destination vertex
 - If graph G contains 'n' vertices then matrix is **$n \times n$**

Adjacency matrix representation

- Directed graph

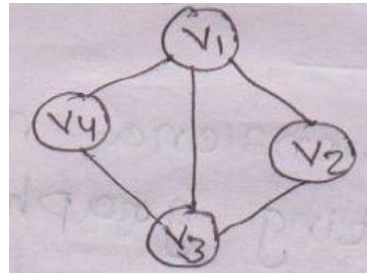
In directed graph there is an edge between V_i and V_j then $M[i][j]=1$. If there is no edge then $M[i][j]=0$.



	V_1	V_2	V_3	V_4
V_1	0	1	0	0
V_2	0	0	1	0
V_3	1	0	0	1
V_4	1	0	0	0

- Undirected graph

In undirected graph there is an edge between V_i and V_j then $M[i][j]=1$ and $M[j][i]=1$. If there is no edge then $M[i][j]$ and $M[j][i]$ both are zero

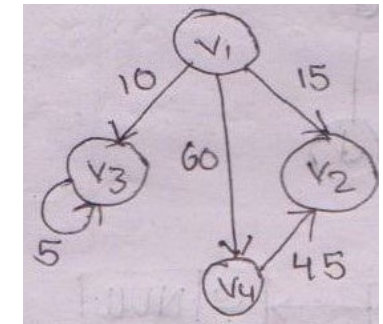


	V_1	V_2	V_3	V_4
V_1	0	1	1	1
V_2	1	0	1	0
V_3	1	1	0	1
V_4	1	0	1	0

Dr. Venkata Phanikrishna B, SCOPE, VIT-Vellore

- Weighted Graph

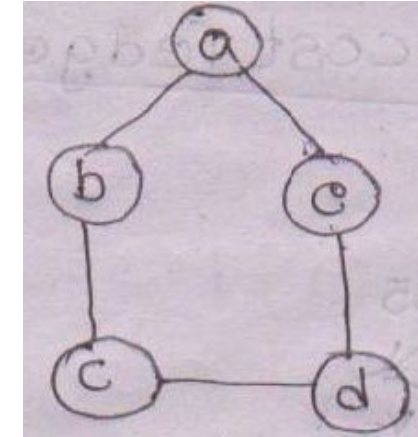
In weighted graph, weight or cost or distance are mentioned on edge. If there is an edge between V_i and V_j then $M[i][j]=\text{cost}[\text{edge}]$.



	V_1	V_2	V_3	V_4
V_1	0	15	10	60
V_2	0	0	0	0
V_3	0	0	5	0
V_4	0	45	0	0

Adjacency list

- An adjacency list of a graph is a collection of unordered list, one for each vertex in the graph.
- It is done by using linked list.
- Each list describes the set of neighbours of its vertex.



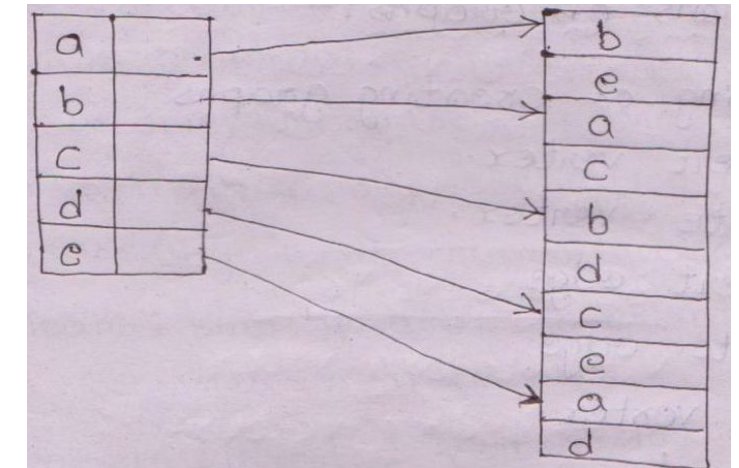
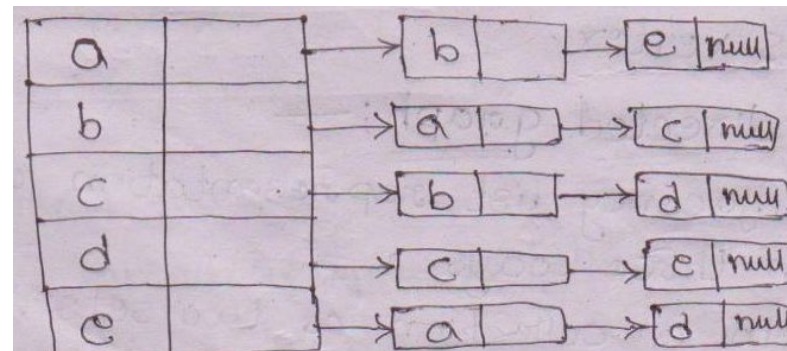
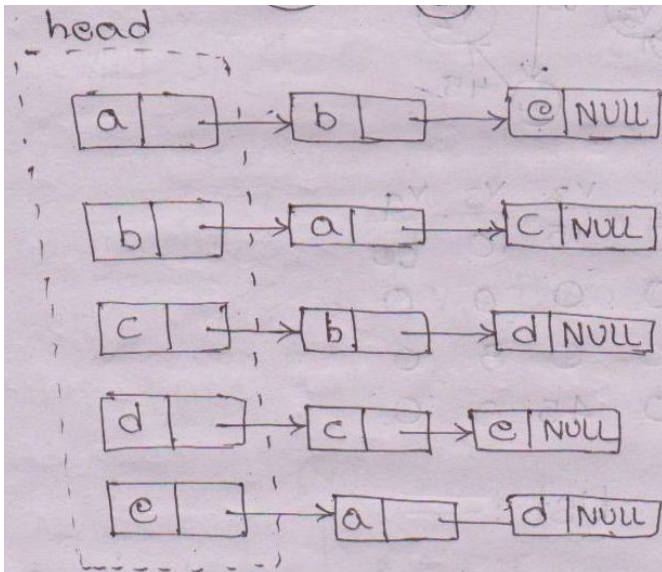
In case of undirected graph, the adjacency list representation is in three ways.

we use an **array of a list** to represent the graph.

The list size is equal to the number of vertex(n).

In this we use **mixed data structure**. I.e., we use **array head** nodes instead of head list as a linked list

In this method adjacency list using **cursor** is used.



<u>Operations</u>	<u>Adjacency Matrix</u>	<u>Adjacency List</u>
Storage Space	$O(V ^2)$	$O(V + E)$
Adding a vertex	$O(V ^2)$. To add a new vertex to $V \times V$ matrix the storage must be increases to $(V +1)^2$	$O(1)$ Using front node and rear node.
Adding an edge	$O(1)$ Direct put value $matrix[i][j] = 1$	$O(1)$ Similar to insertion of vertex.
Removing a vertex	$O(V ^2)$. To remove a new vertex from $V \times V$ matrix the storage must be decrease to $(V -1)^2$.	$O(V + E)$. To remove a vertex, we need to search for the vertex which will require $O(V)$ time in worst case, after this we need to traverse the edges and in worst case it will require $O(E)$ time.
Removing an edge	$O(1)$ Directly make $matrix[i][j] = 0$.	$O(E)$ To remove an edge traversing through the edges is required
Finding vertices (U,V) are adjacent	$O(V)$	$O(1)$

Graph Traversing

- Graph Traversing: Is the process of visiting every node (vertex) exactly once.
- *Graph Traversing is for*
 - Finding a particular node available or not in a given graph.
 - Finding all reachable nodes.
 - Finding best reachable nodes.
 - Finding best path through a graph.
 - Determining whether a graph is DAG (**Directed acyclic graph**: Graph which doesn't contains a cycles)

The main goal of graph traverse is to find all nodes reachable from a given node.

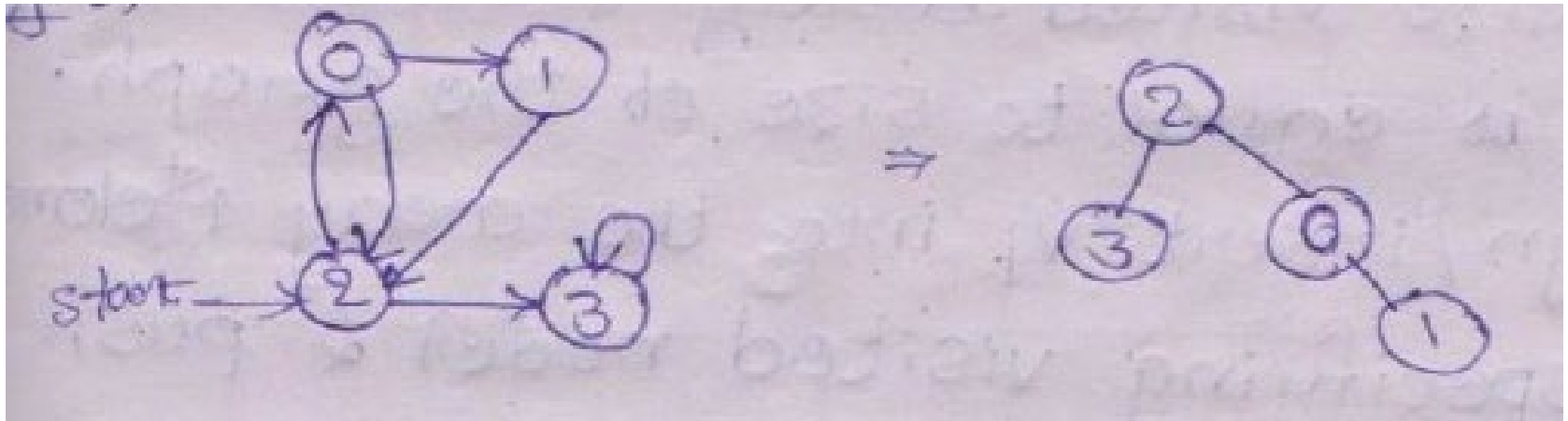
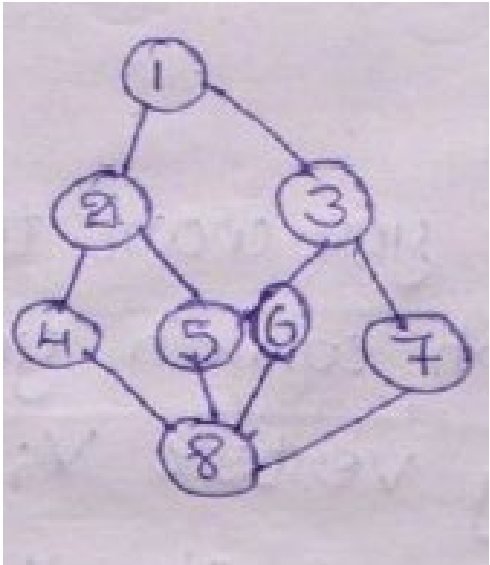
- In undirected graph we follow all edges.
- In directed graph we follow out edge.

Graph traversing strategies

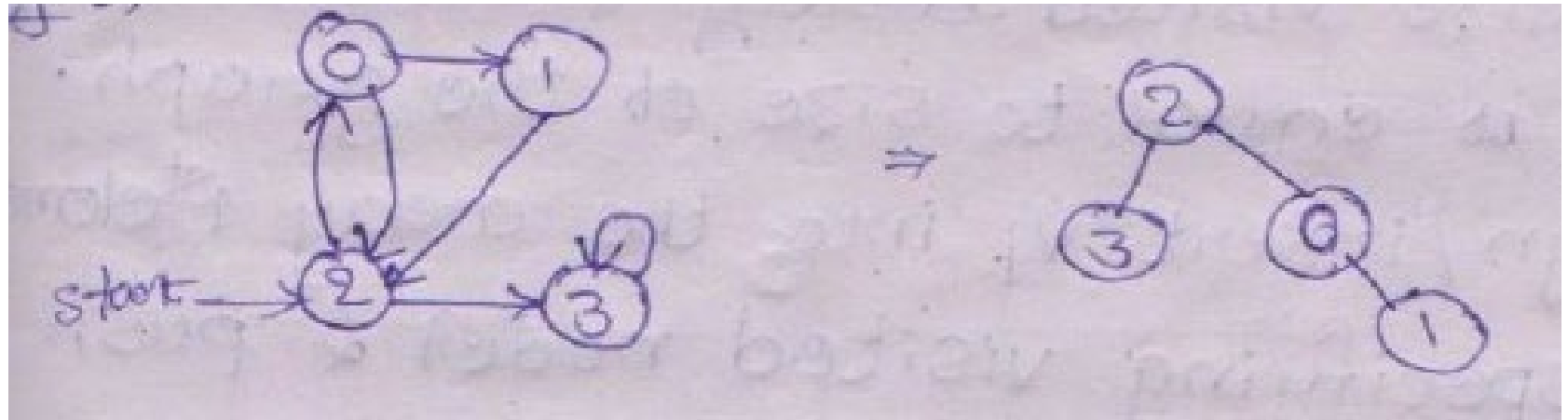
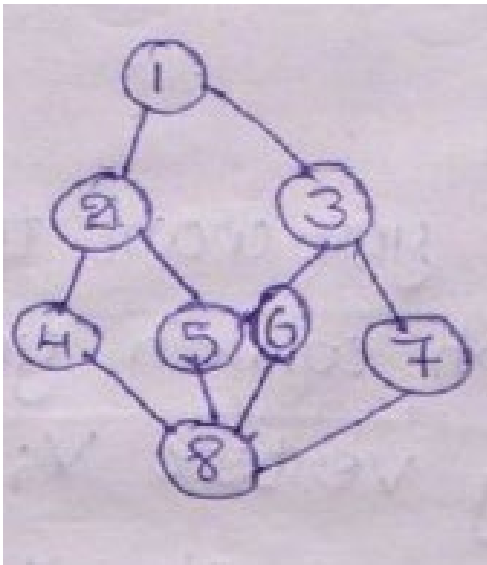
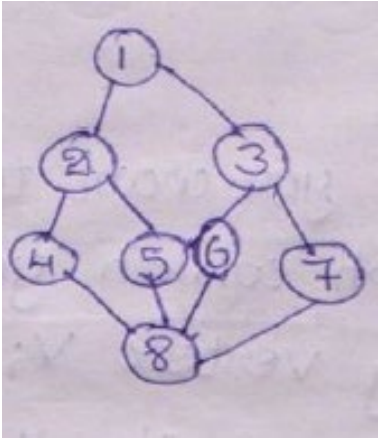
- Graph traversing has two strategies
 - DFS (Depth First Search algorithm)
 - BFS (breadth First Search algorithm)

DFS (Depth First Search algorithm)

- Was investigated in 19th century by a French mathematician, Charles Tremaux for solving Maze
- **Working:** In DFS, start at the root (Selecting some arbitrary node as the root in the graph) and explores (moves/ visit), as far as possible along each branch before back-tracking.



DFS (Depth First Search algorithm)



DFS (Depth First Search) algorithm

- Implementation of DFS: DFS implemented by using **stack data** structures.

Step-1: Consider a graph which you want to traverse.

Step-2: Read vertex from the graph which you want to traverse, say vertex V_i .

Step-3: Initialize visited array and stack. Array size is equal to the size of (order of) the graph (#nodes).

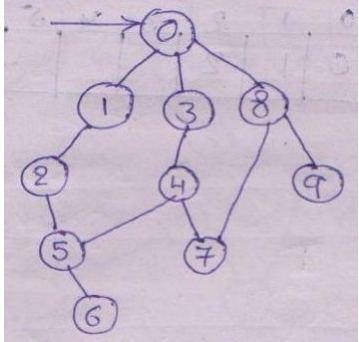
Step-4: Assign/insert V_1 into the array 1st element (for specifying visited node) and push all adjacent nodes/vertices of V_i into the stack.

Step-5: Pop the top value of the stack and insert it into the visited array. And Push all of its adjacent nodes (i.e., popped node/element).

Step-6: If the pop value (Top of the stack) is already present in the array, then don't insert it into the visited array; just discard (or) neglect the top value.

Step-7: Do **Step-5** until the stack is empty and the array is full.

Step-1

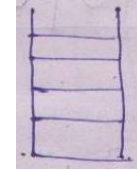
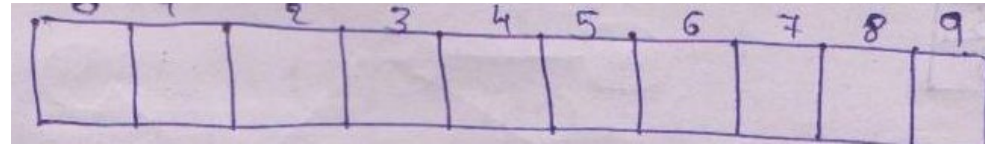


Step-2

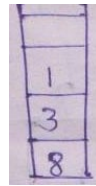
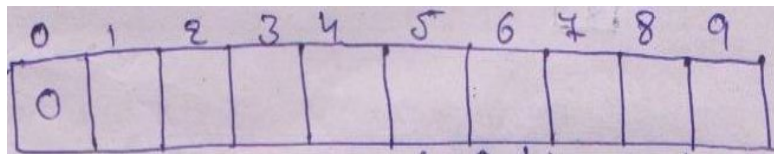
Here $V_i = 0$

Step-3

Visited array (Length of the visited array is the number of nodes in the graph. Take stack, and Initially stack is empty.

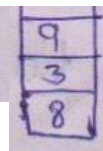
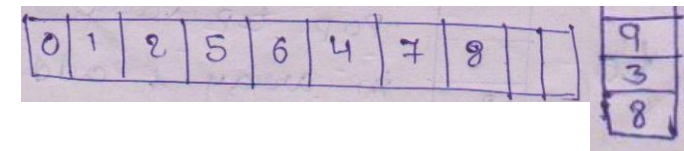
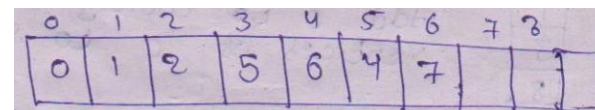
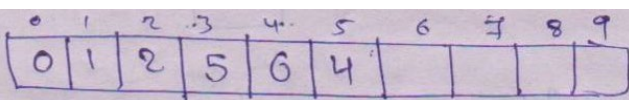
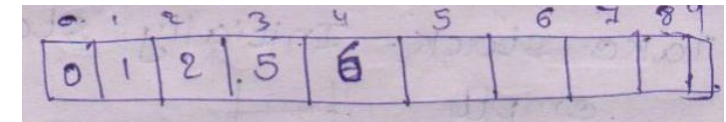
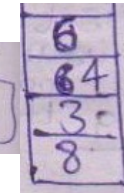
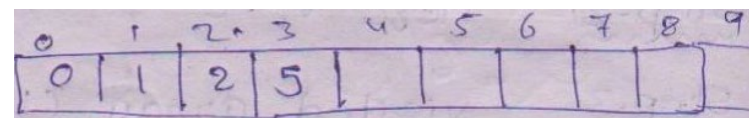
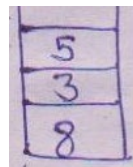
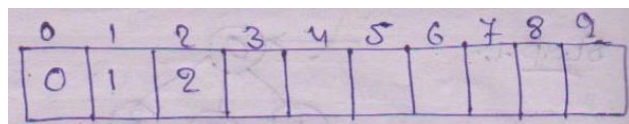
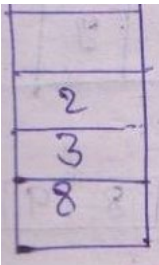
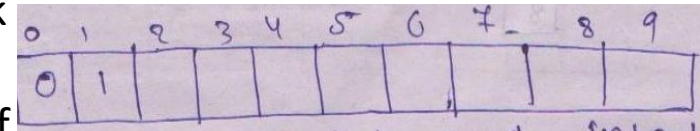


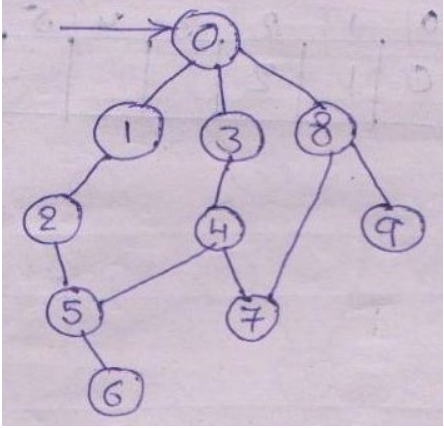
Step-4 Add 0 to array (visited vertex) and PUSH adjacent vertices of 0 to stack (1,3,8)



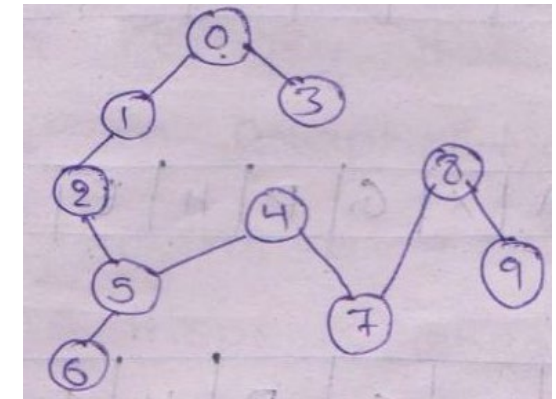
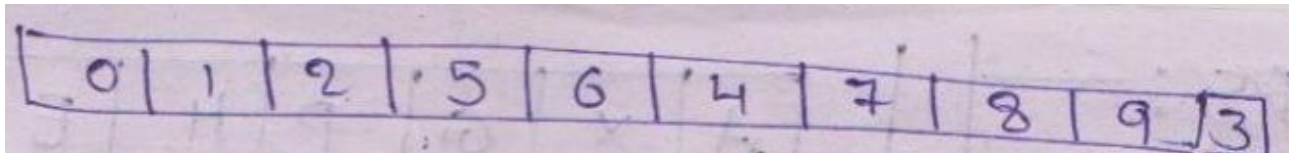
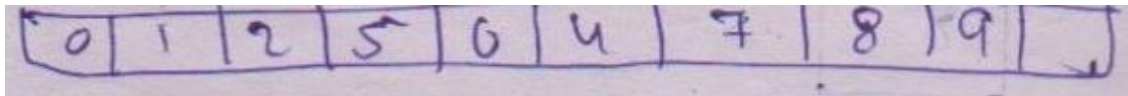
Step-5

POP the top value of the stack (i.e., 1) and insert it into the array. Add adjacent vertices of POPED vertex to the stack.



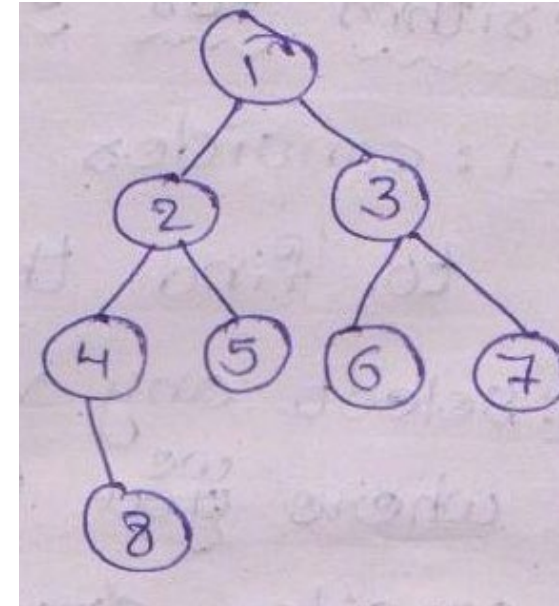
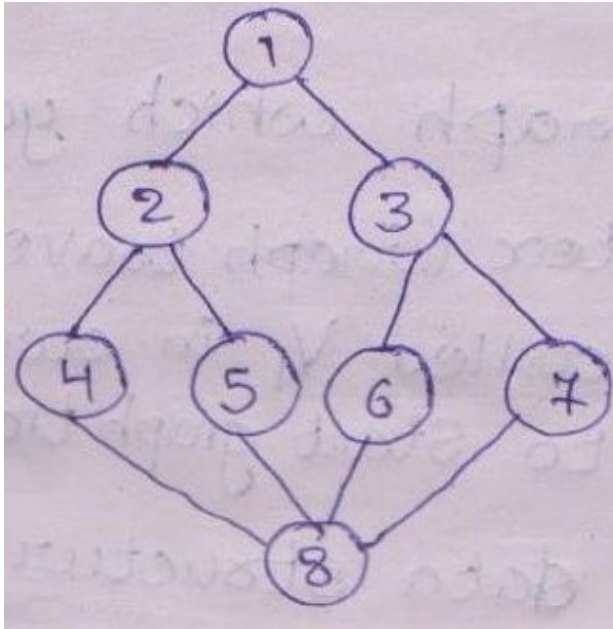


3
8



BFS (Breadth First Search) algorithm

Working: The BFS begins at a particular node or vertex and inspects (observe) all the neighbor nodes. Then, in turn, each of those neighbor nodes inspects their unvisited neighbor nodes and so on.



BFS (Breadth First Search) algorithm

- Implementation of BFS: BFS implemented by using **queue data** structure.

Step-1: Consider a graph which you want to traverse.

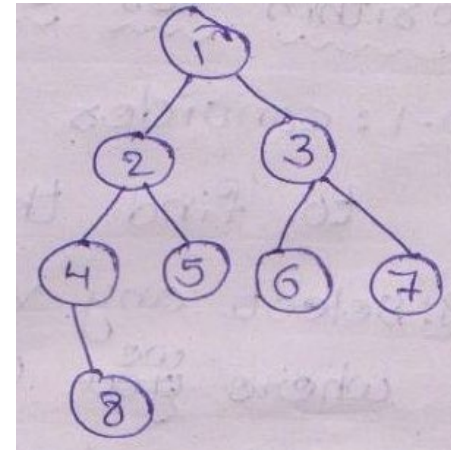
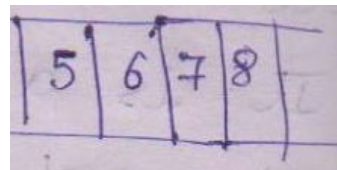
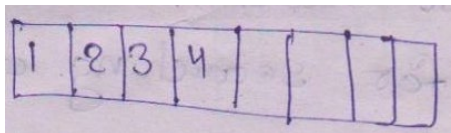
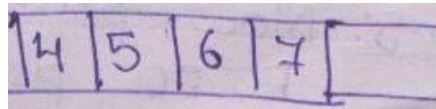
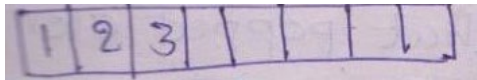
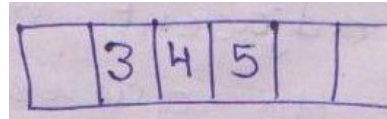
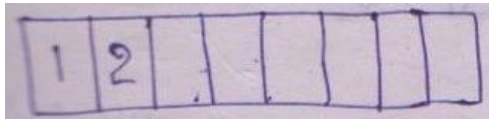
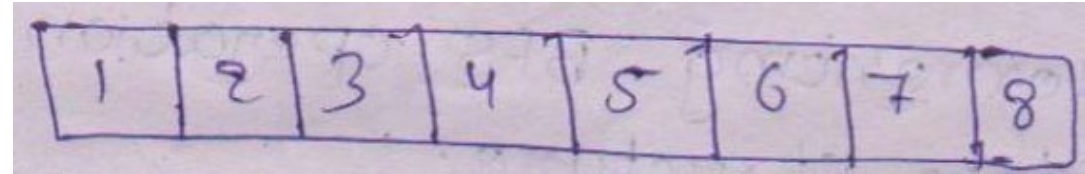
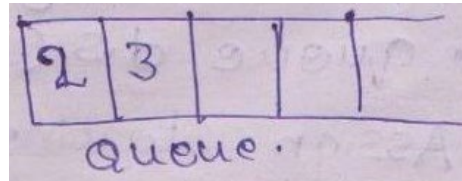
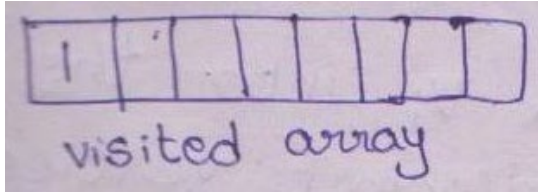
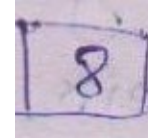
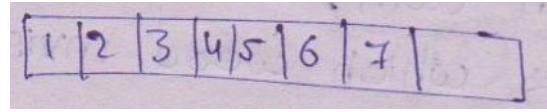
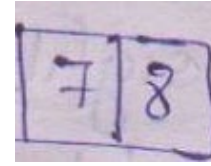
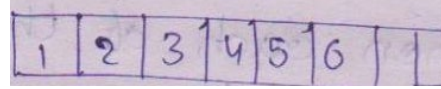
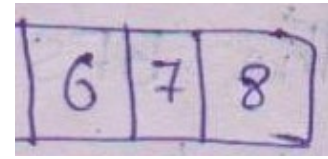
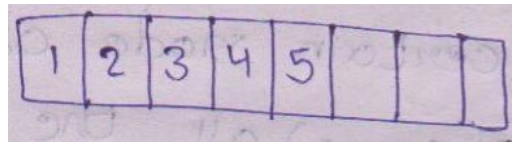
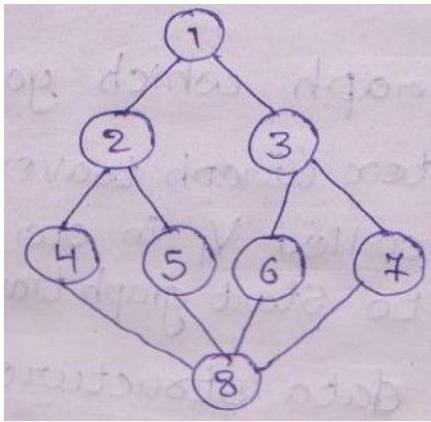
Step-2: Read vertex from the graph which you want to traverse, say vertex V_i .

Step-3: Initialize visited array and Queue data structure (FIFO). Array size is equal to the size of the graph.

Step-4: Assign/insert V_1 into the array 1st element (for specifying visited node) and push all adjacent nodes/vertices of V_i into the Queue.

Step-5: Pop the element of the Queue by using FIFO principle, and insert it into the visited array. And Push all adjacent nodes of popped node/element in to Queue.

Step-6: Do Step-5 until the Queue is empty and the array is full. Must be here is no loop



Any

Question



PresenterMedia

Thank You!

**FOR YOUR
ATTENTION**

